SERVERLESS IMAGE

PROCESSOR

Project implementation and operations report

A detailed description of the image thumbnail application, its browser and local-folder workflows, its simulated AWS architecture, setup procedure, test evidence, and next steps.

PROJECT SNAPSHOT

Interface: Streamlit browser UI

Processing: Python + Pillow in an AWS Lambda-compatible handler

Cloud simulation: LocalStack S3, Lambda, and IAM APIs

Local alternative: command-line folder processor

Current verification: 6 unit tests pass; LocalStack is not running now

Purpose of this report

Use this report as a project explanation for a class or technical presentation. It separates implemented behavior from integration checks that still need to be performed with Docker and LocalStack.

Project repository: serverless-image-processor-Project

1. Executive summary

The project creates smaller, format-preserving image thumbnails. It has two distinct ways to use the processor: a local command-line workflow that reads and writes files in folders, and a browser workflow that uploads through a Streamlit interface to LocalStack S3 and relies on an S3 event to invoke a Lambda function.

The browser flow is a local development demonstration of an event-driven serverless design. LocalStack simulates AWS services on the developer machine; this setup is not an AWS production deployment and does not create real cloud buckets.

KEY DISTINCTION

Browser uploads: input-bucket -> S3 event -> Lambda -> output-bucket

Command line: input_images folder -> local Python processing -> output_images folder

These workflows do not automatically copy files between one another.

2. Goals and project scope

- Accept supported raster image files and generate thumbnails up to 300 x 300 pixels while preserving aspect ratio.
- Keep source format where possible; do not enlarge images already smaller than the target bounds.
- Provide a browser-based upload, preview, and download experience for the LocalStack demonstration.
- Retain a simpler local-folder processor that works without starting LocalStack.
- Package the Lambda handler and Pillow dependency for the Linux Python 3.12 runtime, including when building on Windows.

3. Technology stack

Python 3.10+ is documented for the project. Pillow performs image decoding, orientation handling, resizing, and encoding. Boto3 calls the S3 and Lambda-compatible LocalStack APIs. Streamlit provides the web UI. Docker Compose runs LocalStack and builds the Lambda artifact from the AWS Lambda Python 3.12 base image.

4. Architecture and browser request flow

End-to-end sequence

> Browser user selects a file and presses Generate Thumbnail.
> app.py reads the uploaded bytes in memory and creates a unique uploads/<id>_<name> key.
> Boto3 PutObject stores the original in LocalStack input-bucket.
> LocalStack S3 emits ObjectCreated; the configured notification invokes Lambda.
> lambda_handler.py reads the original object and calls shared image_processor.py code.
> Lambda writes the thumbnail under the matching thumbnail key to output-bucket.
> app.py polls that exact key (up to 30 seconds), then shows preview and download.

The UI computes the expected output key using thumbnail_object_key. The Lambda uses the same helper, so the source object key and generated thumbnail key follow a shared naming convention. A random prefix in each upload key prevents uploaded files with the same original filename from colliding.

Browser interaction and data handling

- The app defaults to LocalStack at http://localhost:4566, input-bucket and output-bucket; environment variables can override the app endpoint and bucket names.
- The upload is sent as bytes with its content type. The UI keeps the returned thumbnail bytes in Streamlit session state for display and download.
- The download button serves the thumbnail held by the app; it does not copy files into output_images.
- The sidebar displays endpoint and bucket names as configuration information; it is not a live health check.

5. Local-folder workflow

Running image_processor.py directly invokes the separate command-line path. By default, it scans files directly in input_images and writes named thumbnails into output_images, creating the output folder if needed. The folders are relative to the current working directory and can be overridden with --input and --output.

```
.\.venv\Scripts\python.exe image_processor.py
.\.venv\Scripts\python.exe image_processor.py --input "C:\photos" --output "C:\thumbs"
```

The CLI skips unsupported file extensions and does not scan subfolders. The image extension list in image_processor.py includes BMP, GIF, JPEG/JPG, PNG, TIFF/TIF, and WebP.

6. Main files and responsibilities

Application modules

- app.py - Streamlit page; validates and previews the selected image, uploads it to S3, waits for the result, and offers a download.
- image_processor.py - shared thumbnail operations and CLI entry point; defines accepted extensions, thumbnail naming, and local-folder processing.
- lambda_handler.py - Lambda entry point; reads S3 event records, downloads each supported original, generates a thumbnail, and stores it in the output bucket.
- setup_localstack.py - creates input/output buckets as needed, deploys or updates Lambda, grants S3 invocation permission, and configures the bucket notification.
- Dockerfile.lambda - uses the AWS Lambda Python 3.12 base image and installs Pillow into the deployment asset directory.
- package_lambda.py - writes the contents of the asset directory to build/lambda.zip.
- docker-compose.yml - defines LocalStack and the one-shot Lambda package build service.
- test_local.py - automated unit tests for image resizing, file workflow validation, and naming helpers.

7. Image-processing behavior

- The target thumbnail bound is 300 x 300 pixels; Pillow preserves aspect ratio when resizing.
- EXIF orientation is applied before resizing. JPEG images with incompatible modes are converted to RGB for saving.
- The thumbnail name adds _thumbnail before the lowercased file extension. For example, photo.JPG becomes photo_thumbnail.jpg.
- The Lambda derives the thumbnail object key from the original key, preserving its folder prefix.
- Non-supported file types are skipped by the Lambda and the local directory scanner.

8. Setup and run instructions (PowerShell)

Prerequisites

Install Python 3.10 or newer and Docker Desktop with Compose, start Docker Desktop, and open PowerShell in the project root. Dependencies are Pillow, boto3, and streamlit; a virtual environment is recommended.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

8. Setup and run instructions (continued)

Start the LocalStack browser demonstration

```
docker compose up -d localstack
docker compose ps
docker compose run --build --rm lambda-package
python setup_localstack.py
python -m streamlit run app.py
```

Wait for the localstack service to report healthy before running setup_localstack.py. The package build writes build\lambda.zip. The setup script then creates or updates the buckets, Lambda function, invoke permission, and S3 event notification. Streamlit prints the local browser URL after starting.

Test the local-folder processor

```
python -m unittest -v test_local
python image_processor.py
```

Stop services

```
docker compose down
```

The LocalStack data directory is .localstack and persists simulated AWS state. Stopping the containers does not imply that bucket contents have been deleted. Resetting LocalStack data is destructive; only do so when the stored demo data is no longer needed.

9. Verification and current status

OBSERVED CHECKS AT REPORT PREPARATION

Automated tests: 6 test_local.py tests passed.

The Streamlit UI rendered in an AppTest smoke check without an app exception.

The Lambda deployment ZIP exists at build/lambda.zip (about 7.6 MB).

docker compose ps showed no running Compose services at the time checked.

The unit tests cover local image processing and helper behavior; they are not integration tests for S3 event delivery or Lambda execution. Since LocalStack was not running at the time of inspection, the browser upload -> S3 -> Lambda -> S3 -> browser result was not freshly verified end to end for this report.

10. Test coverage and limitations

The six existing tests exercise: creating a thumbnail from bytes; thumbnail dimensions for a large image; leaving a smaller image at its original size; ignoring a non-image file; handling JPEG filename extension case; deriving an S3 thumbnail key; rejecting the same input and output folder; and rejecting a missing input directory. Some of these behaviors are combined in individual test methods.

The test suite does not mock boto3 in test_local.py, start LocalStack, assert Lambda deployment responses, verify an S3 bucket notification, test browser upload actions, or test production AWS permissions. The UI has been smoke-tested for rendering, but the live LocalStack event chain needs a separate run.

11. Security and deployment boundary

- The configured endpoint, bucket names, and test credentials target local development. Do not treat these credentials as production secrets or use them against real AWS.
- The repository setup provisions simulated services on the local Docker network. There is no public hosting, authentication, user management, or production IAM deployment described by this implementation.
- For production, define an AWS account/region, least-privilege IAM roles and bucket policies, secret/configuration management, network exposure, authentication, upload size/type limits, monitoring, retention, and error handling.
- The S3 buckets are intended for the LocalStack demonstration. The UI upload and output are not written to input_images/output_images.

12. Recommended next steps for completion

- Start Docker Desktop and verify docker compose ps reports LocalStack healthy.
- Build the Lambda ZIP and run setup_localstack.py; inspect setup output and LocalStack logs for failures.
- Start Streamlit, upload a small supported image, and verify preview and download in the browser.
- Confirm the original exists in input-bucket and the matching _thumbnail object exists in output-bucket; inspect Lambda logs if the UI times out.
- Repeat with a JPEG and PNG, then a small image and an unsupported extension to check expected behavior.
- If production hosting is a goal, treat it as a distinct deployment phase and design real AWS infrastructure and controls before exposing the application.

13. Short presentation summary

This project demonstrates two ways to generate image thumbnails. The local Python workflow processes files in folders. The browser workflow uses Streamlit to upload to LocalStack S3, trigger a Lambda-compatible processor, and preview or download the result. The image processing and unit tests are implemented; the remaining validation for the browser demo is a fresh end-to-end run with LocalStack active. LocalStack is a simulation, so production use would require real AWS deployment and security work.